

TRABALHANDO COM ATRIBUTOS E MÉTODOS

Prof. André Backes

Encapsulamento

- Em várias situações, o acesso direto aos atributos de um objeto não é aconselhável
 - Pode comprometer a lógica e funcionamento da classe
 - Altera dados do objeto sem verificação apropriada

Encapsulamento

- O encapsulamento se refere à ideia de esconder os detalhes de implementação de um objeto ou classe
 - Conceito fundamental da programação orientada a objetos
 - Fica exposto apenas os métodos e atributos necessários para o funcionamento do sistema
 - Permite que o código seja mais modular e fácil de manter

Encapsulamento vs TAD

Tipo Abstrato de Dados

- Define um conjunto de dados e as operações permitidas sobre eles
 - Não fornece informação sobre a implementação desses dados e operações
 - Abstração da estrutura de dados e suas operações
 - Não suporta recursos mais sofisticados, como a herança

Encapsulamento

- Busca esconder os detalhes de implementação de um objeto
 - Proteger os dados e as operações de acesso e manipulação diretos
 - Permite controlar quem pode acessar os membros de uma classe
 - Faz uso de modificadores de acesso

Encapsulamento vs TAD

- A abstração fornecida pelo TAD permite que os usuários usem o tipo de dado sem precisar se preocupar com os detalhes de implementação, enquanto o encapsulamento garante que esses detalhes sejam protegidos
 - TAD é uma especificação que define o que um tipo de dado faz
 - Encapsulamento é um mecanismo que protege a implementação do TAD

Funções de acesso

- Método público utilizado para recuperar ou alterar o valor de uma variável membro privada de uma classe
 - Fundamentais para o princípio de encapsulamento
 - Protege os dados contra acessos e modificações diretas por código externo
 - Apenas as interfaces públicas da classe são acessíveis externamente

Funções de acesso

- Garantem a manipulação controlada dos dados, respeitando as regras internas
 - Por exemplo, idade negativa
- Dois tipos principais:
 - **Getter**: Recupera o valor de um atributo privado
 - **Setter**: Modifica o valor de um atributo privado

```
class Pessoa {
private:
    string nome;
    int idade;
public:
    // Função de acesso para o nome
    string getNome() const { return nome; }
    // Função de acesso para modificar o nome
    void setNome(string n) { nome = n; }
    // Função de acesso para idade
    int getIdade() const { return idade; }
    // Função de acesso para modificar a idade
    void setIdade(int i) {
        if (i >= 0) { // Evita idades negativas
            idade = i;
        }
    }
};

int main() {
    Pessoa p;
    p.setNome("Bob");
    p.setIdade(30);
    cout << "Nome: " << p.getNome() << endl;
    cout << "Idade: " << p.getIdade() << endl;

    return 0;
}
```

Funções de acesso

- Não há uma convenção de nomenclatura
- Formas populares:
 - Usar os prefixos **get** e **set**
 - Sobrecarga de função
 - Usar o prefixo **set** no modificador

```
class Data{  
private:  
    int dia, mes, ano;  
public:  
    // Com prefixos get and set  
    int getAno() { return ano; }  
    void setAno(int A) { ano = A; }  
  
    // Sem prefixos (sobrecarga de função)  
    int Mes() { return mes; }  
    void Mes(int m) { mes = m; }  
  
    // Com prefixo apenas no modificador  
    int Dia() { return dia; }  
    void setDia(int d) { dia = d; }  
};
```


Funções de acesso

- Nunca crie funções de acesso sem sentir uma real necessidade por elas
 - Muitos deles nunca serão invocados
 - Muitos poderiam e deveriam ser substituído por métodos de negócios
 - Como sacar se não tiver saldo?

```
class Conta {  
private:  
    double limite, saldo;  
public:  
    double getSaldo() { return saldo; }  
    void setSaldo(double _s) { saldo = _s; }  
    double getLimite() { return limite; }  
    void setLimite(double _l) { limite = _l; }  
};
```

Funções de acesso

- Uma ideia melhor seria reformular a classe e limitar o número de funções de acesso
 - Podemos acessar o saldo, mas sua alteração depende das regras de negócio

```
class Conta {  
private:  
    double saldo, limite;  
public:  
    Conta(double _l, double _s): limite {_l}, saldo{_s}{}  
    void deposita (double x) {  
        saldo += x;  
    }  
  
    void saca(double x) {  
        if(saldo + limite >= x) {  
            saldo -= x;  
        } else  
            throw "Erro: estourou limite!";  
    }  
    double getSaldo() {  
        return saldo;  
    }  
};
```

Concedendo o acesso com friend

- Atributos privados não podem ser acessados por funções externas a classe
 - Porém, em algumas situações isso pode ser necessário
 - Mas, ao mesmo tempo, não queremos que o atributo seja público

Concedendo o acesso com friend

- A palavra-chave **friend** permite conceder acesso especial a partes privadas da classe
 - Usado com funções ou outras classes
 - Permite acesso membros privados (ou protegidos) sem recorrer a outras soluções, como a herança

Funções usando friend

- Permite o acesso a atributos privados de uma classe por uma função externa
 - O protótipo da função precisa ser inserido dentro da classe

```
class ContaBancaria {  
    private:  
        double saldo;  
  
    public:  
        void definirSaldo(double s) {saldo = s; }  
        double obterSaldo() {return saldo; }  
        friend void print(const ContaBancaria& c);  
};  
  
void print(const ContaBancaria& c) {  
    cout << "Saldo: " << c.saldo << endl;  
}  
  
int main() {  
    ContaBancaria conta;  
    conta.definirSaldo(1000.50);  
  
    print(conta);  
  
    return 0;  
}
```

Funções usando friend

- É especialmente útil quando precisamos acessar membros privados de duas classes
 - A função é declarada como **friend** em ambas as classes
 - Note que o protótipo da classe Pessoa deve ser declarado antes de ContaBancaria

```
class Pessoa;  
class ContaBancaria {  
    private:  
        double saldo;  
  
    public:  
        void definirSaldo(double s) {saldo = s; }  
        double obterSaldo() {return saldo; }  
        friend void print(const ContaBancaria& c, const Pessoa& p);  
};  
  
class Pessoa {  
    private:  
        string nome;  
  
    public:  
        void definirNome(string n) { nome = n;}  
        friend void print(const ContaBancaria& c, const Pessoa& p);  
};  
  
void print(const ContaBancaria& c, const Pessoa& p){  
    cout << "Nome: " << p.nome << endl;  
    cout << "Saldo: " << c.saldo << endl;  
}
```

Classes usando friend

- Uma classe **friend** é uma classe que recebe permissão para acessar os membros privados e protegidos de outra classe
 - Permite uma colaboração mais estreita entre duas classes, sem comprometer completamente o encapsulamento

Classes usando friend

- Declarar a classe como **friend**
 - Permite que uma classe acesse diretamente os membros privados da outra classe
 - Particularmente útil quando uma classe precisa acessar diretamente os dados internos de outra classe para fornecer funcionalidades específicas
 - Aqui, Pessoa tem acesso a parte privada de ContaBancaria

```
class ContaBancaria {  
    private:  
        double saldo;  
    public:  
        void definirSaldo(double s) {saldo = s; }  
        double obterSaldo() {return saldo; }  
        friend class Pessoa;  
};  
  
class Pessoa {  
    private:  
        string nome;  
    public:  
        void definirNome(string n) { nome = n;}  
        void exibirSaldo(const ContaBancaria& c){  
            cout << "Nome: " << nome << endl;  
            cout << "Saldo: " << c.saldo << endl;  
        }  
};  
  
int main() {  
    ContaBancaria conta;  
    conta.definirSaldo(1000.50);  
  
    Pessoa p;  
    p.definirNome("Carlos");  
    p.exibirSaldo(conta);  
    return 0;  
}
```


Classes usando friend

- O comando **friend** define que uma classe terá acesso aos atributos privados de outra classe, nada além disso
 - **Pessoa** acessa o conteúdo privado de **ContaBancaria**, mas não o contrário
 - Classes derivadas de Pessoa não tem acesso a **ContaBancaria**

Classes usando friend

- Ao invés de toda a classe, podemos tornar uma única função como **friend**
 - Melhor, mas mais complicado
- Existe toda uma questão envolvendo
 - A visibilidade de uma classe pela outra
 - A ordem de declaração das classes
 - A ordem de declaração dos protótipos

Classes usando friend

- Melhor, mas mais complicado
 - Declarar o protótipo de ContaBancaria antes de Pessoa
 - Definir o protótipo da função em Pessoa
 - Faz referência a ContaBancaria
 - Definir o protótipo da função em ContaBancaria
 - Explícito que se trata de uma função de outra classe e como **friend**
 - Implementação de exibirSaldo()
 - Fora das duas classes para garantir a visibilidade por ambas

```
class ContaBancaria;
class Pessoa {
private:
    string nome;
public:
    void definirNome(string n) { nome = n;}
    void exibirSaldo(const ContaBancaria& c);
};

class ContaBancaria {
private:
    double saldo;
public:
    void definirSaldo(double s) {saldo = s; }
    double obterSaldo() {return saldo; }
    friend void Pessoa::exibirSaldo(const ContaBancaria& c);
};

void Pessoa::exibirSaldo(const ContaBancaria& c){
    cout << "Nome: " << nome << endl;
    cout << "Saldo: " << c.saldo << endl;
}

int main() {
    ContaBancaria conta;
    conta.definirSaldo(1000.50);

    Pessoa p;
    p.definirNome("Carlos");
    p.exibirSaldo(conta);
    return 0;
}
```

Ponteiro this

- Ponteiro especial presente em todas as classes em C++
 - Está disponível dentro dos métodos não estáticos de uma classe
 - Utilizado para referenciar o próprio objeto que invocou o método
 - Permite acesso direto aos seus membros
 - Facilita diversas operações na programação orientada a objetos

Ponteiro this

- Ponteiro implícito
 - Aponta para o próprio objeto que chamou o método
 - Permite acesso aos seus próprios atributos e métodos de maneira explícita
 - Muito útil quando se deseja diferenciar o atributo do parâmetro em um método, evitando ambiguidades

```
class Pessoa {  
    private:  
        string nome;  
        int idade;  
  
    public:  
        void setNome(string nome) {  
            // uso explícito  
            this->nome = nome;  
        }  
  
        void setIdade(int age) {  
            // uso implícito  
            idade = age;  
        }  
};
```

Ponteiro this

- Deixa explícita a manipulação de atributos de objetos diferentes
 - Comparar o valor de um atributo presente em dois objetos da mesma classe

```
class Comparador {  
    private:  
        int numero;  
  
    public:  
        Comparador(int n) : numero(n) {}  
  
        bool ehMaiorQue(const Comparador* outro) {  
            return this->numero > outro->numero;  
        }  
};  
  
int main() {  
    Comparador obj1(10), obj2(5);  
  
    if (obj1.ehMaiorQue(&obj2))  
        cout << "obj1 é maior que obj2" << endl;  
  
    return 0;  
}
```

Encadeamento de métodos

- Permite que várias funções da classe possam ser chamadas no mesmo objeto em uma única expressão
 - Um uso interessante do ponteiro **this**
- Um exemplo é o **cout**
 - Podemos fazer múltiplas chamadas do operador << e exibir diversas variáveis em sequência

```
cout << X << Y << Z;
```

Encadeamento de métodos

- Sem encadeamento
 - Métodos são do tipo **void**
 - São chamados em sequência no programa principal

```
class Contador {  
    private:  
        int valor = 0;  
  
    public:  
        void incrementar() {  
            valor++;  
        }  
  
        void mostrar() {  
            cout << "Valor: " << valor << endl;  
        }  
};  
  
int main() {  
    Contador c;  
  
    c.incrementar();  
    c.incrementar();  
    // Saída: Valor: 2  
    c.mostrar();  
  
    return 0;  
}
```


Encadeamento de métodos

- Com encadeamento
 - Retorna um ponteiro para o próprio objeto está sendo manipulado, **this**
 - Permite o encadeamento de sucessivas chamadas no programa principal

```
class Contador {  
    private:  
        int valor = 0;  
  
    public:  
        Contador& incrementar() {  
            valor++;  
            // Retorna a referência ao próprio objeto  
            return *this;  
        }  
  
        void mostrar() {  
            cout << "Valor: " << valor << endl;  
        }  
};  
  
int main() {  
    Contador c;  
    // Saída: Valor: 2  
    c.incrementar().incrementar().mostrar();  
  
    return 0;  
}
```

Encadeamento de métodos

- Com encadeamento
 - Cada chamada de **incrementar()** retorna um ponteiro **this**
 - Esse ponteiro pode ser usado para fazer uma nova chamada a um método da classe

```
c.incrementar().incrementar().mostrar();  
(    this    ).incrementar().mostrar();  
(          this          ).mostrar();
```

Encadeamento de métodos

- O ponteiro **this** é útil na sobrecarga de operadores
 - Especialmente com operadores matemáticos
 - Podemos querer concatenar várias operações matemáticas
 - Garante a execução correta da sobrecarga do operador

```
class Contador {  
private:  
    int valor;  
public:  
    Contador(int v = 0): valor {v}{}  
  
    Contador& operator++() {  
        // Pré incremento (++valor)  
        ++valor;  
        return *this;  
    }  
    Contador operator++(int) {  
        // Pós incremento (valor++)  
        Contador temp = *this;  
        ++valor;  
        return temp;  
    }  
    Contador& operator--() {  
        // Pré decremento (--valor)  
        --valor;  
        return *this;  
    }  
    Contador operator--(int) {  
        // Pós decremento (valor--)  
        Contador temp = *this;  
        --valor;  
        return temp;  
    }  
};
```

MÉTODOS E ATRIBUTOS ESTÁTICOS

Tipos de atributos

- Atributos de instância
 - Cada objeto possui sua própria cópia dos atributos definidos na classe
 - São exclusivos de cada objeto
- Atributos de classe (estático)
 - São compartilhados por todas as instâncias (objetos) de uma classe
 - Pertencem à classe e não a instâncias individuais
 - Existe apenas uma cópia desse atributo

Atributo estático

- Usa a palavra-chave **static** na declaração do atributo
 - A definição e a inicialização são feitas fora da classe
 - **totalObjetos** é compartilhada entre todas as instâncias da classe **Contador**

```
class Contador {  
public:  
    // Declaração do atributo estático  
    static int totalObjetos;  
    Contador() {  
        totalObjetos++;  
    }  
};  
// Definição do atributo estático  
int Contador::totalObjetos = 0;
```

Atributo estático

- Acesso
 - Diretamente pelo nome da classe
 - Enfatiza a natureza compartilhada do atributo estático
 - Através de um objeto da classe

```
class Contador {  
public:  
    // Declaração do atributo estático  
    static int totalObjetos;  
    Contador() {  
        totalObjetos++;  
    }  
};  
// Definição do atributo estático  
int Contador::totalObjetos = 0;  
  
int main() {  
    Contador c1;  
    Contador c2;  
    Contador c3;  
  
    cout << "Total: " << Contador::totalObjetos << endl;  
    cout << "Total: " << c1.totalObjetos << endl;  
    return 0;  
}
```

E:\teste.exe

Total: 3

Total: 3

Atributo estático

- Alterar o valor do atributo estático afeta todos os objetos
 - Mesmo usando um objeto da classe
 - O atributo pertence a classe e não ao objeto

```
class Produto{
public:
    static double preco;
};

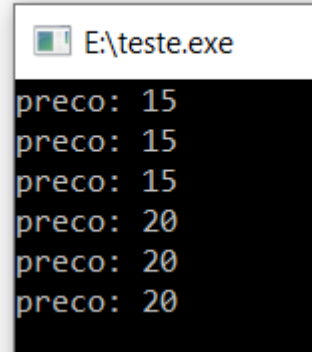
double Produto::preco = 15;

int main() {
    Produto p1, p2;

    cout << "preco: " << Produto::preco << endl;
    cout << "preco: " << p1.preco << endl;
    cout << "preco: " << p2.preco << endl;

    //altera o preço para todos
    p1.preco = 20;
    cout << "preco: " << Produto::preco << endl;
    cout << "preco: " << p1.preco << endl;
    cout << "preco: " << p2.preco << endl;

    return 0;
}
```



Atributo estático

- Inicialização é um inconveniente
 - Normalmente é feita fora da classe
- Solução: atributo constante
 - Permite inicializar o atributo estático dentro da definição de classe
 - **const**: tipos inteiros (como **int**, **char** e **bool**)
 - **constexpr** (padrão C++11): **double**

```
class Produto{  
public:  
    static const int qtd = 10;  
    static constexpr double preco = 15;  
};
```

Métodos estáticos

- Podem ser usados sem criar um objeto
 - Implementado dentro ou fora da classe
 - Não é necessário utilizar novamente a palavra reservada **static** se a implementação for fora da classe
 - Útil para acessar atributos estáticos definidos como privados

```
class Util {  
private:  
    static const int valor = 10;  
public:  
    static void mensagem() {  
        cout << "Este é um método estático." << endl;  
    }  
  
    static int getValor();  
};  
  
int Util::getValor() {  
    return Util::valor;  
}  
  
int main() {  
    Util::mensagem();  
    cout << Util::getValor() << endl;  
  
    return 0;  
}
```

Métodos estáticos | Observações

Não possuem o ponteiro **this**

- O ponteiro **this** sempre aponta para o objeto no qual o método está trabalhando
- Como não existe um objeto associado, não há ponteiro **this**

Não podem ser usados para definir construtores

- Atributos estáticos pode ser inicializado diretamente, sem necessidade de um construtor
- Assim, o C++ não suporta construtores estáticos

MÉTODOS E OBJETOS CONSTANTES

Métodos e objetos constantes

- Modificador **const**
 - Garante a imutabilidade de variáveis, objetos e membros de uma classe
 - Maior segurança ao código
 - Evita modificações indesejadas
 - Permite otimizações por parte do compilador

Objeto constante

- Não pode ter seu estado alterado após a inicialização
 - Permite apenas o uso de métodos declarados como constantes
 - Métodos constantes garantem não modificar os dados do objeto
 - **imprime()** não pode ser usado mesmo que não altere nenhum atributo

```
class Produto {  
private:  
    double preco;  
public:  
    Produto(double p) : preco(p) {}  
  
    double getPreco() const {  
        return preco;  
    }  
  
    void imprime() {  
        cout << "Preco: " << preco << endl;  
    }  
};  
  
int main() {  
    // Objeto constante  
    const Produto p(50.0);  
  
    // Permitido: método é const  
    double valor = p.getPreco();  
  
    // ERRO: não é possível modificar um objeto constante  
    // p.imprime();  
  
    return 0;  
}
```

Ponteiros e constantes

- Ao invés de um objeto constante, podemos querer tornar apenas o ponteiro para esse objeto constante
 - Permite controlar tanto a modificabilidade do objeto apontado quanto a do próprio ponteiro
 - Tudo depende da posição do modificador **const**

Ponteiro constante para objeto mutável

- O modificador **const** é colocado entre o tipo e o nome do ponteiro
 - Ponteiro constante para objeto mutável
 - O ponteiro não pode ser alterado, mas o objeto sim
 - Podemos
 - Acessar todos os métodos
 - Mas não podemos mudar para qual objeto o ponteiro aponta

```
int main() {  
    Produto p1(50.0), p2(100.0);  
  
    Produto* const ptr = &p1;  
  
    // Permitido  
    double valor = ptr->getPreco();  
    ptr->imprime();  
  
    // Erro: ptr é constante  
    // ptr = &p2;  
  
    return 0;  
}
```


Ponteiro mutável para um objeto constante

- O modificador **const** é colocado antes do tipo do ponteiro
 - Ponteiro mutável para objeto constante
 - O ponteiro pode ser alterado, mas o objeto não
 - Podemos
 - Acessar os métodos **const**
 - Não podemos alterar o objeto via ponteiro
 - Podemos mudar para qual objeto o ponteiro aponta

```
int main() {  
    Produto p1(50.0), p2(100.0);  
  
    const Produto* ptr = &p1;  
  
    // Permitido  
    double valor = ptr->getPreco();  
    ptr = &p2;  
    valor = ptr->getPreco();  
  
    // ERRO: não é possível modificar um objeto constante  
    // ptr->imprime();  
  
    return 0;  
}
```

Ponteiro constante para um objeto constante

- Modificador **const** antes tipo e do nome do ponteiro

- Ponteiro constante para objeto constante

- Nem o ponteiro e nem o objeto podem ser alterados

- Podemos

- Acessar os métodos **const**
 - Objeto não pode ser alterado via ponteiro
 - Proibido mudar para qual objeto o ponteiro aponta

```
int main() {  
    Produto p1(50.0), p2(100.0);  
  
    const Produto* const ptr = &p1;  
  
    // Permitido  
    double valor = ptr->getPreco();  
  
    // ERRO: não é possível modificar um objeto constante  
    // ptr->imprime();  
    // ptr = &p2;  
  
    return 0;  
}
```

Parâmetros e retorno de função são constantes

- O objeto é tratado como **const** dentro da função
 - Apenas métodos constantes podem ser usados na função
 - Evita modificações acidentais e permitir otimização pelo compilador

```
void exibir(const Produto& p) {  
    // Permitido  
    cout << "Preço: " << p.getPreco() << endl;  
  
    // ERRO: não é possível modificar um objeto constante  
    // p.imprime();  
}  
  
const Produto& maior(const Produto& p1, const Produto& p2) {  
    return (p1.getPreco() > p2.getPreco()) ? p1 : p2;  
}  
  
int main() {  
    Produto p1(50.0), p2(100.0);  
    Produto p = maior(p1, p2);  
    exibir(p);  
  
    return 0;  
}
```

Material Complementar

- Aula 25 - POO: Funções de Acesso
 - <https://youtu.be/B-vOskA7abE>
- Aula 26 - POO: Funções Amigas (friend)
 - <https://youtu.be/p--k3GXVblE>
- Aula 28 - POO: Ponteiro this
 - <https://youtu.be/rm-ZuXy-dcg>
- Aula 29 - POO: Atributos Estáticos
 - <https://youtu.be/uAncvX92CBA>
- Aula 30 - POO: Métodos e Objetos Constantes
 - <https://youtu.be/Uo2L3PDaACs>